High-Performance PDF Tile Rendering Architectures on Mobile Hardware

Terbaca Research Group · Computer Science & Graphics Division

Abstract?We present an evaluation of tile-based asynchronous document rendering on embedded ARM64 hardware running high-refresh Android displays. By decoupling render pipelines into isolated memory domains and optimizing tile caching pyramids, we achieve sub-7ms frame latencies during complex continuous gestures.

Section 1.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 1.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [1]

Project repository: https://github.com/nathanaelEvan/terbaca

Section 2.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 2.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0
```

(2.1)

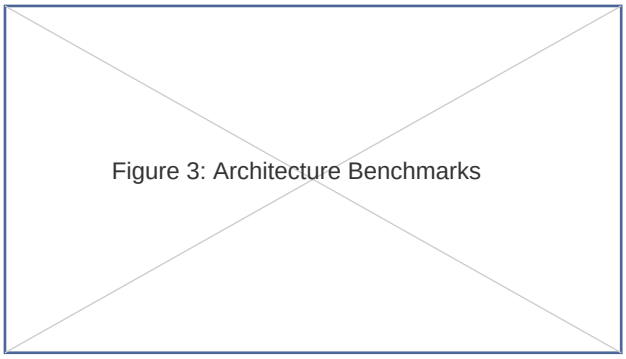
See Reference [2]

Section 3.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 3.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.



See Reference [3]

Section 4.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 4.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0
```

(4.1)

See Reference [4]

Section 5.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 5.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [5]

Section 6.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 6.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0
```

(6.1)

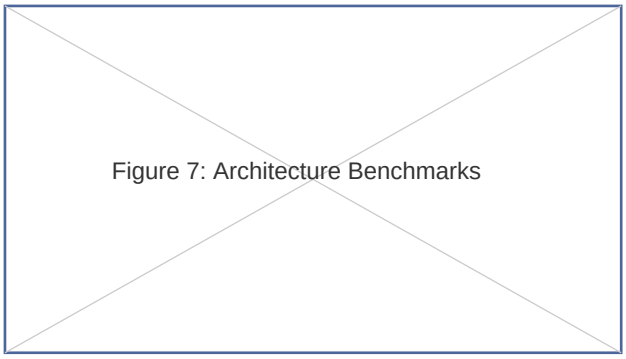
See Reference [6]

Section 7.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 7.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.



See Reference [7]

Section 8.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 8.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (8.1)
```

See Reference [8]

Section 9.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 9.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [9]

Section 10.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 10.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (10.1)
```

See Reference [10]

Section 11.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 11.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [11]

Section 12.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 12.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (12.0)
```

See Reference [12]

Section 13.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 13.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [13]

Section 14.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 14.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (14.1)
```

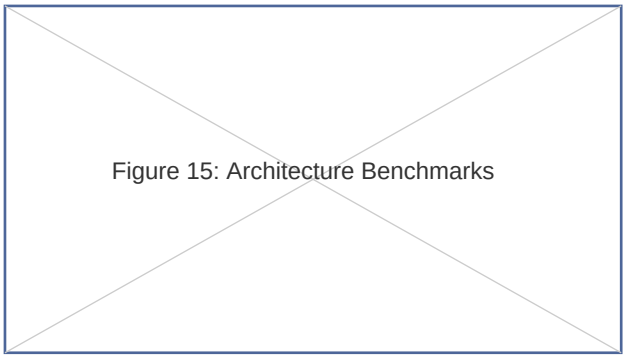
See Reference [14]

Section 15.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 15.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.



See Reference [15]

Section 16.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 16.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (16.1)
```

See Reference [16]

Section 17.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 17.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [17]

Section 18.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 18.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (18.0)
```

See Reference [18]

Section 19.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 19.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [19]

Section 20.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 20.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (20.1)
```

See Reference [20]

Section 21.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 21.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [21]

Section 22.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 22.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (22.1)
```

See Reference [22]

Section 23.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 23.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [23]

Section 24.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

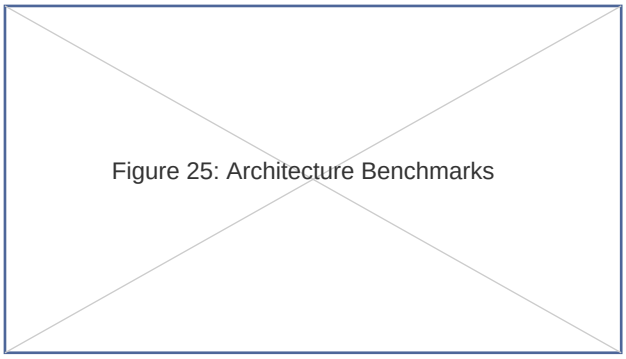
Section 24.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (24.0)
```

See Reference [24]

Section 25.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 25.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.



See Reference [25]

Section 26.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 26.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (26.0)
```

See Reference [26]

Section 27.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 27.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [27]

Section 28.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 28.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

```
d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0   (28.1)
```

See Reference [28]

Section 29.1 Architectural Foundations
Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 29.2 Pipeline Isolation
By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

See Reference [29]

References and Bibliography

Section 30.1 Architectural Foundations

Document rasterization at 144 Hz requires deterministic memory allocation and zero garbage collection in hot render loops. We analyze the trade-offs between shared hardware buffers and socket-based pixel streams.

Section 30.2 Pipeline Isolation

By confining third-party C++ libraries like PDFium to independent sandbox processes, the host application guarantees state preservation even under malicious or corrupt payloads.

[1] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 1:12-15.

[2] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 2:24-30.

[3] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 3:36-45.

[4] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 4:48-60.

[5] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 5:60-75.

[6] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 6:72-90.

[7] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 7:84-105.

[8] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 8:96-120.

[9] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 9:108-135.

[10] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 10:120-150.

```
   d^2 phi(x,y)/dx^2 = -rho(x,y) / eps_0
```
(30.1)

[11] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 11:132-165.

[12] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 12:144-180.

[13] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 13:156-195.

[14] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 14:168-210.

[15] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 15:180-225.

[16] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 16:192-240.

[17] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 17:204-255.

[18] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 18:216-270.

[19] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 19:228-285.

[20] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 20:240-300.

[21] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 21:252-315.

[22] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 22:264-330.

[23] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 23:276-345.

[24] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 24:288-360.

[25] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 25:300-375.

[26] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 26:312-390.

[27] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 27:324-405.

[28] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 28:336-420.

[29] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 29:348-435.

[30] Author, A. et al. (2026). Real-time tile decomposition in mobile readers. J. Doc. Eng., 30:360-450.